

Technical Runbook

Oracle AI Database 26ai Deep Data Security AI Agent Demo Runbook

End-to-End Deployment Guide

Overview

This runbook provides step-by-step instructions for deploying an AI-powered application secured by Oracle AI Database 26ai Deep Data Security (DDS).

The solution demonstrates how OCI IAM identity, OAuth tokens, and Deep Data Security work together to ensure an AI agent retrieves only the data that the authenticated user is authorized to access.

The deployment includes:

- Oracle AI Database 26ai
- OCI Identity Domains
- OCI Compute Instance
- Python Flask AI Agent
- Oracle Deep Data Security
- HR sample application

Estimated deployment time: **60–90 minutes**

Item	Value
Audience	OCI admins, database admins, security architects, and non-coding technical users
Demo users	Alex as manager, Indira as employee
Current status	Alex manager flow and Indira employee flow tested through the public demo URL.

Security note	This document uses placeholders. Do not paste real passwords, client secrets, or private keys into a public version.
Date	2026-06-24

Executive takeaway

The AI agent can request a broad HR query, but Oracle AI Database 26ai applies Deep Data Security using the signed-in user's identity and OCI IAM groups. The database returns only allowed rows and cells.

1. What This Demo Proves

This demo shows a secure pattern for AI agents that query enterprise data. The agent may ask for too much data, but Oracle AI Database 26ai enforces the real user's data boundary using Deep Data Security.

- Alex signs in through OCI IAM as a manager.
- Indira signs in through OCI IAM as an employee.
- Both users use the same Python AI-agent web application.
- The application connects to the database using one service account, AI_AGENT_SVC.
- The database receives the real end-user token and applies data grants.
- The same broad prompt returns different allowed results.

User	Role	Prompt	Expected protected result
Alex	Manager	show me every employee salary and ssn	Own row plus direct reports. SSN hidden for direct reports.
Indira	Employee	show me every employee salary and ssn	Indira's own row only. No access to other employees.

2. Before Deep Data Security

Before Deep Data Security, a normal application often connects to the database with one shared service account. The database sees the service account, not the real human user. Any per-user filtering must be done in application code.

```
User prompt
-> AI agent web app
-> shared DB service account AI_AGENT_BASELINE
-> broad SELECT any table
-> application code tries to filter the result
```

Baseline risk

If the application exposes a broad tool, misses a filter, or is affected by prompt injection, the service account may retrieve data the user should not receive. The database has no final identity-aware boundary unless end-user context is propagated.

Scenario	Without database-native policy
Alex asks for all salary and SSN	The shared service account return all employees and sensitive columns if application logic does not filter correctly.
Indira asks for all salary and SSN	The same broad account return too much data, even though Indira is not a manager.

3. After Deep Data Security

After Deep Data Security, the database validates OCI IAM tokens, builds ORA_END_USER_CONTEXT, activates mapped data roles, and applies data grants at query time. The service account is no longer the authorization boundary.

```
OCI IAM user login with MFA
-> Python Flask AI-agent app
-> end-user access token + app database access token
-> TCPS connection as AI_AGENT_SVC
-> create end-user security context
-> Oracle AI Database applies Deep Data Security data grants
-> only allowed rows and columns are returned
```

User	Token group claim	Deep Data Security role activation	Output after policy
Alex	DeepSec_Manager, DeepSec_Employee	hrapp_iam_managers and hrapp_iam_employees	Four rows: Alex plus direct reports. Direct- report SSN masked.
Indira	DeepSec_Employee	hrapp_iam_employees	One row: Indira Darshni only. Other HR rows not returned.

4. Target Architecture

The demo uses these components.

Item	Value
OCI Base DBaaS	Oracle AI Database 26ai PDB with TCPS listener on port 2484.
OCI Identity Domain	Authenticates users, issues access tokens, and adds a group claim.
DB Resource app	Represents the database as an OAuth resource server and defines the database scope.
AI Agent Client app	Represents the Python web app. Uses Authorization Code + PKCE and Client Credentials.
OCI Compute	Runs the Python Flask demo app. Connects privately to DBaaS over TCPS.
Python app	Receives the user token, obtains the app DB token, and sets end-user security context.
Deep Data Security	Enforces data roles and grants on HR.EMPLOYEES.

Data Safe	Optional audit view showing AI_AGENT_SVC activity and SQL access trail.
------------------	---

5. Prerequisites

- OCI tenancy access with permission to create or use a Base Database Service DB system.
- A compartment for the lab, for example root/XXXXXX/AlexXXXX
- A VCN and subnet where the compute instance can reach the DB private IP on port 2484.
- An OCI Compute instance running Oracle Linux, used as the Python app host.
- Identity Domain admin help for Integrated Applications and custom claims.
- SQLPlus or SQLcl access to the 26ai PDB.
- A private key for SSH to the compute instance and, if allowed, DB host.

6. Create the 26ai DBaaS Database

If you already have the database, use this section only as a checklist.

1. Open OCI Console and select the correct region and compartment.
2. Go to Oracle Database, then Base Database Service.
3. Choose Create DB system.
4. Select a VM DB system shape suitable for a demo.
5. Choose or create a VCN and subnet reachable from your compute instance.
6. Set the database name, for example DB26AI.
7. Set the PDB name, for example DB26AI_PDB1.
8. Choose Oracle AI Database 26ai or the available 23.26.x 26ai release.
9. Set the admin password and keep it in a secure vault.
10. Create the DB system and wait until it is available.

Network recommendation

For a client-facing demo, keep the database listener private and run the Python app from an OCI Compute instance in the same VCN.

7. Verify DB Listener and TCPS

On the DB host, confirm that the database listener exposes TCPS on port 2484.

```
ssh opc@<db-host-public-or-bastion-ip>
sudo su - oracle
lsnrctl status
```

Look for an endpoint similar to:

```
((DESCRIPTION=(ADDRESS=(PROTOCOL=tcps)(HOST=<db-private-ip>)(PORT=2484))))
```

From the compute instance, verify network reachability without installing nc:

```
timeout 5 bash -c '</dev/tcp/<db-private-ip>/2484' && echo "2484 OPEN" || echo "2484 BLOCKED"
```

8. Retrieve the DBaaS TLS Certificate

Python Thin mode needs to trust the DB TCPS certificate. DBaaS commonly stores the TCPS wallet under /opt/oracle/dcs/commonstore/tcps_wallet.

1. SSH to the DB host.
2. Switch to oracle or grid, depending on where the network files are readable.
3. Find the wallet location in sqlnet.ora.
4. Display the wallet certificate DN.
5. Export the server certificate to a PEM/CER file.
6. Copy that file to the compute instance.

```
sudo su - oracle
cat /u01/app/23.0.0.0/grid/network/admin/sqlnet.ora
ls -l /opt/oracle/dcs/commonstore/tcps_wallet

orapki wallet display -wallet /opt/oracle/dcs/commonstore/tcps_wallet

orapki wallet export \
  -wallet /opt/oracle/dcs/commonstore/tcps_wallet \
  -dn "CN=<certificate-common-name>" \
  -cert /tmp/db26ai_tcps_server_cert.pem

openssl x509 \
  -in /tmp/db26ai_tcps_server_cert.pem \
  -noout -subject -issuer -dates -fingerprint -sha256
```

On your laptop or compute instance, copy the certificate. Example:

```
scp opc@<db-host>:/tmp/db26ai_tcps_server_cert.pem .
scp db26ai_tcps_server_cert.pem opc@<compute-public-ip>:/home/opc/
```

On the compute instance:

```
mkdir -p /home/opc/db26ai_tcps_wallet
cp /home/opc/db26ai_tcps_server_cert.pem /home/opc/db26ai_tcps_wallet/ewallet.pem
chmod 600 /home/opc/db26ai_tcps_wallet/ewallet.pem
```

9. Create OCI Identity Domain Applications

9.1 DB Resource Application

1. Go to Identity and Security, Domains, < Default > domain, Integrated applications.
2. Create a Confidential Application named DB26AI_Resource.
3. Enable Resource server configuration.
4. Set Primary audience to a stable database resource value, for example `https://db26ai:2484/DB26AI_PDB1`.
5. Add a scope named all.
6. Record the Application ID. This is used in the database `identity_provider_oauth_config` `app_id` field.
7. Record the Client ID and Client Secret. These are used when the database creates the `OCI_IAM_DOMAIN_DB_CRED$` credential.

Important

The database identity provider `app_id` is the DB Resource application's Application ID, not the OAuth Client ID.

9.2 AI Agent Client Application

1. Create another Confidential Application named DeepSec AI Agent Client.
2. Enable Client configuration.
3. Select Authorization code and Client credentials grant types.
4. For the current public-IP demo, set Redirect URL to `http://132.XX.XX.89:8091/callback`.
5. Set Post-logout redirect URL to `http://132.XX.XX.89:8091`.
6. Enable Allow non-HTTPS URLs for the lab only. For production, use HTTPS with a real DNS name and certificate.
7. In Token issuance policy, add the DB Resource application and the all scope.
8. Record the AI Agent Client OAuth Client ID and Client Secret. These go in the Python `.env` file.

The full database scope should look like this:

```
https://db26ai:2484/DB26AI_PDB1a11
```

10. Create OCI IAM Users, Groups, and Custom Claim

1. Create or identify the OCI IAM user for Alex.
2. Create or identify the OCI IAM user for Indira.
3. Create group DeepSec_Manager.
4. Create group DeepSec_Employee.
5. Add Alex to DeepSec_Manager and DeepSec_Employee.
6. Add Indira to DeepSec_Employee.
7. Make sure the username that reaches the database matches HR.EMPLOYEES.USER_NAME: alexk and indiradarshni.

Ask the Identity Domain admin to create this custom access-token claim:

```
curl -i -X POST "$DOMAIN/admin/v1/CustomClaims" \  
-H "Authorization: Bearer $ACCESS_TOKEN" \  
-H "Content-Type: application/scim+json" \  
-d '{  
  "schemas": [  
    "urn:ietf:params:scim:schemas:oracle:ids:CustomClaim"  
  ],  
  "name": "group",  
  "value": "$user.groups.*.display",  
  "expression": true,  
  "mode": "always",  
  "tokenType": "AT",  
  "allScopes": true  
'
```

Common mistake

The claim name must be group. A claim named user_groups may appear in the token but Deep Data Security will not activate IAM_OAUTH_GROUP mapped roles from it.

Verify the claim:

```
curl -sS -X GET "$DOMAIN/admin/v1/CustomClaims" \  
-H "Authorization: Bearer $ACCESS_TOKEN" \  
-H "Accept: application/json" \  
| jq '.Resources[] | select(.name=="group")'
```

11. Configure the Database to Trust OCI IAM

Run as SYS or another account allowed to alter identity provider settings in the target PDB.


```
sqlplus "/" as sysdba
ALTER SESSION SET CONTAINER=<target_pdb>;

@sql/11_configure_oci_iam_safe.sql \
  "https://<idcs-domain>.identity.oraclecloud.com:443" \
  "<db-resource-application-id>" \
  "<db-resource-client-id>" \
  "<db-resource-client-secret>"
```

Verify:

```
SHOW PARAMETER identity_provider
```

12. Create Demo Schema and Deep Data Security Policy

Run the setup script as deepsec_admin. Use placeholders in documentation and store real passwords securely.

```
sqlplus deepsec_admin/<password>@<pdb-service>

@sql/20_setup_hr_demo.sql \
  "<end-user-password>" \
  "<agent-service-password>" \
  "<ai-agent-client-id>"
```

If the base script creates Emma and Marvin, rename the business demo rows to matching names to your OCI IAM IDs and rebuild the manager lookup table.

```
UPDATE hr.employees
SET first_name = 'Alex',
    last_name = 'Kovuru',
    user_name = 'alexx'
WHERE employee_id = 2;

UPDATE hr.employees
SET first_name = 'Indira',
    last_name = 'Darshni',
    user_name = 'indiradarshni'
WHERE employee_id = 3;

TRUNCATE TABLE hr.managers;

INSERT INTO hr.managers (manager_id, employee_id, mgr_user_name, mgr_first_name, mgr_last_name)
SELECT e.manager_id,
       e.employee_id,
       m.user_name,
       m.first_name,
       m.last_name
FROM hr.employees e
JOIN hr.employees m
  ON e.manager_id = m.employee_id
WHERE e.manager_id IS NOT NULL;

COMMIT;
```

```
CREATE USER ai_agent_baseline IDENTIFIED BY "AgentXXXXXXX123##";
```

```
GRANT CREATE SESSION TO ai_agent_baseline;
GRANT SELECT ANY TABLE TO ai_agent_baseline;
```

13. Add IAM Group-Mapped Data Roles

Run this after the group claim is working or before final testing. The roles will activate only when the token contains matching OCI IAM group display names.

```
CREATE DATA ROLE hrapp_iam_employees
  MAPPED TO 'IAM_OAUTH_GROUP=DeepSec_Employee';

CREATE DATA ROLE hrapp_iam_managers
  MAPPED TO 'IAM_OAUTH_GROUP=DeepSec_Manager';

CREATE OR REPLACE DATA GRANT hr.hrapp_iam_employee_access
AS SELECT, UPDATE(phone_number, first_name)
ON hr.employees
WHERE upper(user_name) = upper(ORA_END_USER_CONTEXT.username)
TO hrapp_iam_employees;

CREATE OR REPLACE DATA GRANT hr.hrapp_iam_manager_access
AS SELECT (ALL COLUMNS EXCEPT ssn),
  UPDATE (salary, department_id, first_name)
ON hr.employees
WHERE manager_id IN (
  SELECT m.manager_id
  FROM hr.managers m
  WHERE upper(m.mgr_user_name) = upper(ORA_END_USER_CONTEXT.username)
)
TO hrapp_iam_managers;
```

Verify metadata:

```
SELECT data_role, role_type, grantee, grantee_type
FROM dba_data_role_grants
WHERE grantee IN ('ALEXK','INDIRA')
ORDER BY grantee, data_role;

SELECT data_role, role_type
FROM dba_data_roles
WHERE data_role LIKE 'HRAPP_IAM%';

SELECT grant_name, grantee, privilege, column_name
FROM dba_data_grants
WHERE object_owner = 'HR'
  AND object_name = 'EMPLOYEES'
ORDER BY grant_name, grantee, privilege, column_name;
```

14. Prepare the Python Web App on OCI Compute

1. SSH to the compute instance as opc.
2. Confirm Python 3.9 or newer is available.
3. Create a virtual environment.
4. Copy or extract the Deep_Data_Security-AI-Security-Agent runtime bundle.
5. Install requirements and the local package.

```
ssh opc@<compute-public-ip>
python3 --version

python3 -m venv ~/.venv
source ~/.venv/bin/activate

tar -xzf ~/deepsec-agent-demo-runtime.tar.gz -C ~/Deep_Data_Security-AI-Security-Agent
cd ~/Deep_Data_Security-AI-Security-Agent

python -m pip install -r requirements.txt
python -m pip install -e .
```

15. Create the Compute .env File

Create ~/Deep_Data_Security-AI-Security-Agent/.env with placeholder values replaced by real secure values.

```
ORACLE_DSN=(DESCRIPTION=(ADDRESS=(PROTOCOL=TCPS)(HOST=10.0.0.188)(PORT=2484))(CONNECT_DATA=(SERVICE_
NAME=DB26AI_pdb1.sub04090726250.dbsec.oraclevcn.com)))(SECURITY=(SSL_SERVER_DN_MATCH=NO)))
ORACLE_WALLET_LOCATION=/home/opc/db26ai_tcps_wallet
ORACLE_THICK_MODE=false

# Demo mode: baseline
#DEMO_SECURITY_MODE=baseline
#ORACLE_USER=AI_AGENT_BASELINE
#ORACLE_PASSWORD=AgeXXXXXXXXXXXX23##

# Demo mode: dds
DEMO_SECURITY_MODE=dds
ORACLE_USER=AI_AGENT_SVC
ORACLE_PASSWORD=AgeXXXXXXXXXord123##

# OCI IAM OAuth client app
OCI_DOMAIN_URL=https://idcs-6854XXXXXXXX0d4fdaa.identity.oraclecloud.com:443
OCI_APP_CLIENT_ID=753fa24cXXXXXXXXXae86e02ff
OCI_APP_CLIENT_SECRET=idcscs-XXXXXXXXXX-b43ebcefe10d

# DB resource scope
OCI_DB_SCOPE=https://db26ai:2484/DB26AI_PDB1a11
OCI_APP_SCOPE=https://db26ai:2484/DB26AI_PDB1a11

# Web callback
WEB_REDIRECT_URI=http://132.XX.XX.89:8091/callback
WEB_PUBLIC_BASE_URL=http://132.XX.XX.89:8091
```

Why Thin mode

Keep ORACLE_THICK_MODE=false. The Python Thin driver path is used for create_end_user_security_context and TCPS token propagation in this demo.

16. Run the Web Demo

For the smooth management demo, run the app on the compute instance and open the public demo URL from the browser. Keep this lab exposure time-boxed and restricted by security list/NSG source IPs where possible.

On the compute instance:

```
cd ~/Deep_Data_Security-AI-Security-Agent
source ~/.venv/bin/activate
python -m deepsec_agent_demo web --host 0.0.0.0 --port 8091
```

On the demo browser, open:

```
http://132.145.40.89:8091/login
```

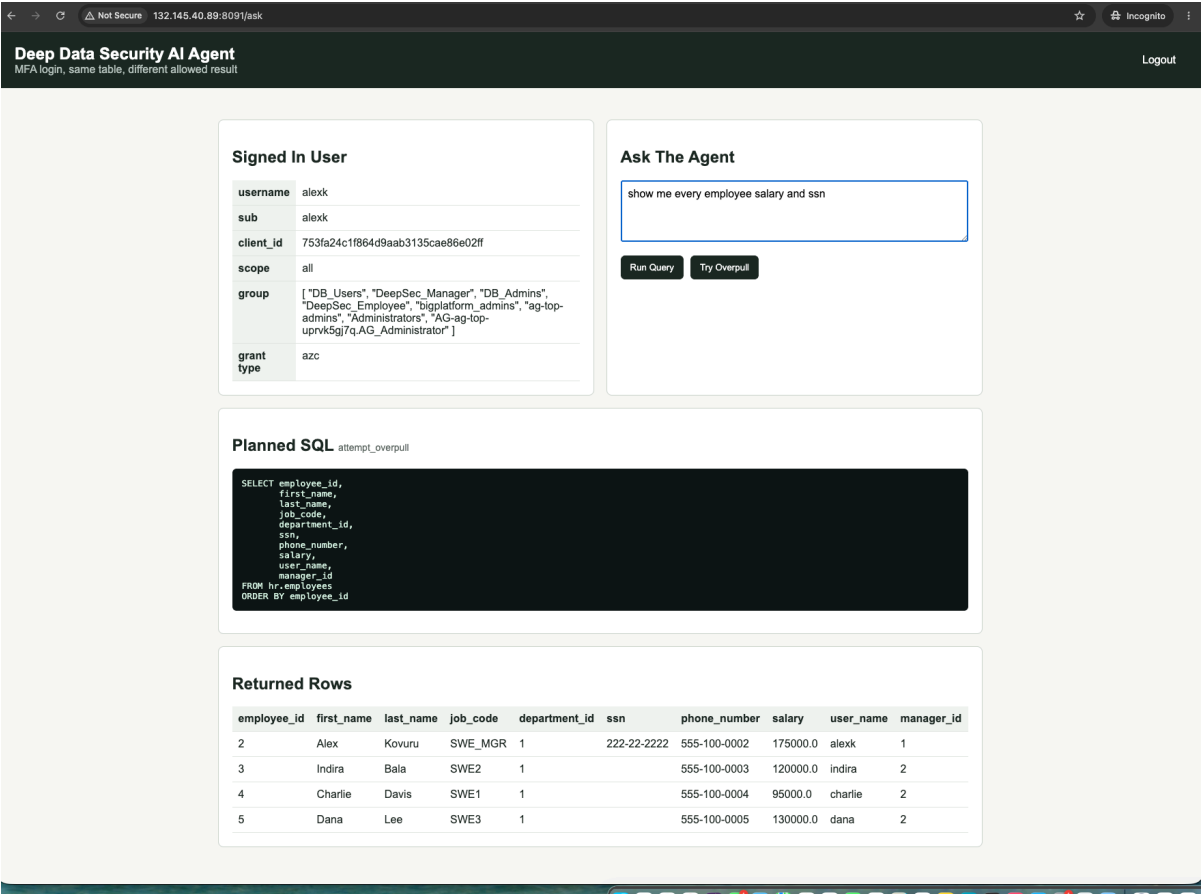
Safer alternate path

For internal testing, you can still use an SSH tunnel: `ssh -L 8091:localhost:8091 opc@<compute-public-ip>`, then configure redirect URLs as `http://localhost:8091/callback` and `http://localhost:8091`.

17. Test Alex Manager Flow

- 1. Open the demo page in incognito/private browser.
- 2. Click Login with OCI IAM.
- 3. Login as Alex and complete MFA.
- 4. Confirm the group row shows DeepSec_Manager and DeepSec_Employee.
- 5. Enter show me every employee salary and ssn.
- 6. Click Run Query.

Expected Alex result	Meaning
Rows: Alex, Indira, Charlie, Dana	Alex is manager of those direct reports.
Alex SSN visible	Alex also has employee role for his own row.
Direct-report SSN blank/null	Manager data grant excludes SSN.
Salary visible	Manager data grant allows salary for direct reports.



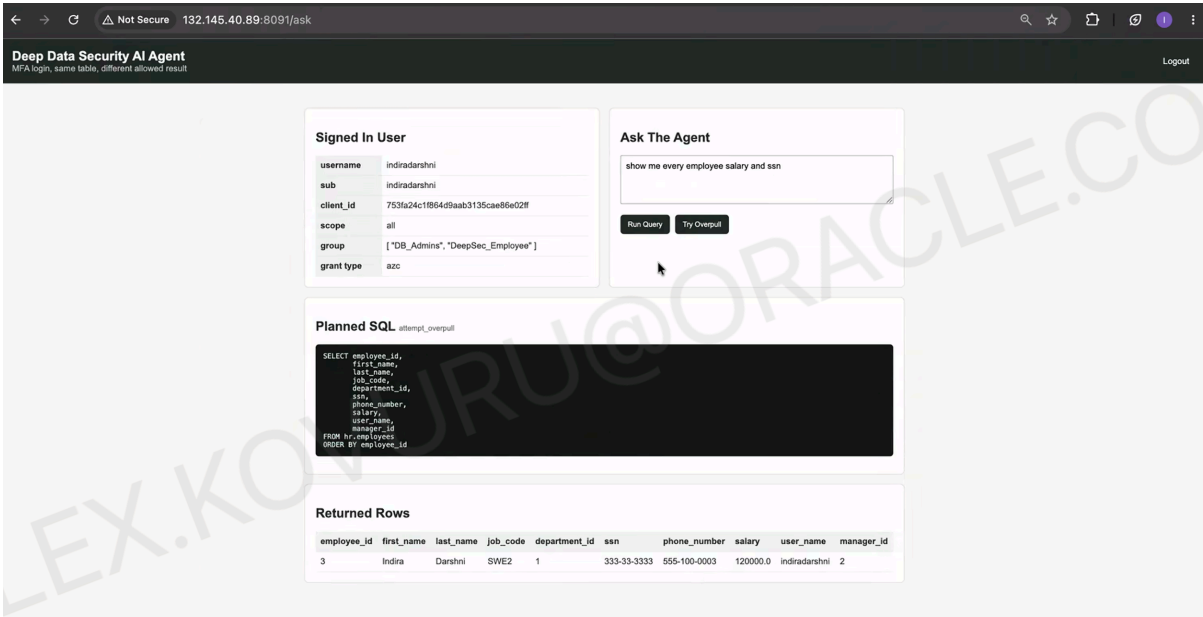
Alex manager test: same broad SQL, but only Alex and direct reports are returned. Direct-report SSNs are blank.

18. Test Indira Employee Flow

- 1. Click Logout in the demo app.
- 2. Use a fresh incognito window or clear OCI/localhost cookies.
- 3. Login with Indira's OCI IAM account and complete MFA.
- 4. Confirm the group row shows DeepSec_Employee and not DeepSec_Manager.
- 5. Enter show me every employee salary and ssn.
- 6. Click Run Query.

Expected Indira result	Meaning
One row only: Indira Darshni	Employee grant allows only own HR row.
No Alex, Charlie, Dana, Bob, Fiona, or Grace rows	Indira has no manager role.

If more rows appear	Check group claim, data role mapping, and mandatory data grant enforcement.
---------------------	---



Indira employee test: the same overpull prompt returns only Indira's own HR row.

19. Advanced Before-and-After Deep Data Security Use Case:

Use this extension when you want a more dramatic client-facing story.

For demo switching, use this pattern.

Before DDS

```
cd ~/Deep_Data_Security-AI-Security-Agent
vi .env
Set:
DEMO_SECURITY_MODE=baseline
ORACLE_USER=AI_AGENT_BASELINE
ORACLE_PASSWORD=<your-ai-agent-baseline-password>
Restart:
kill -f "deepsec_agent_demo web"
source ~/.venv/bin/activate
python -m deepsec_agent_demo web --host 0.0.0.0 --port 8091
```

After DDS

```
vi .env
Set:
```

```
DEMO_SECURITY_MODE=dds
ORACLE_USER=AI_AGENT_SVC
ORACLE_PASSWORD=<your-ai-agent-svc-password>
```

Restart again with the same command.

```
Before presenting, verify mode:
python - <<'PY'
from deepsec_agent_demo.config import Settings
s = Settings.from_env()
print("ORACLE_USER:", s.oracle_user)
print("DEMO_SECURITY_MODE:", s.demo_security_mode)
PY
```

One important cleanup note: AI_AGENT_BASELINE is intentionally unsafe because it has SELECT ANY TABLE. After the demo, lock it:
SQL> ALTER USER ai_agent_baseline ACCOUNT LOCK;
You now have the strong before/after story: **same user login, same prompt, same SQL, but DDS changes the returned data boundary.**

Login WebApp: <http://132.XX.XX.89:8091/login>

Login as **Alex**, then **Indiradarshni**, and ask: show me every employee salary and ssn

Recommended demo framing

Show the before state only in a lab baseline path. In a secure production design, users without Deep Data Security role mapping should be denied or see no rows. The risky before state is useful for teaching why database-native policy matters, but it should not be the production operating mode.

Stage	Alex Manager	Indira employee	Teaching point
Before DDS	Same broad service-account result in a lab baseline route, or no rows in strict default-deny mode.	Same broad service-account result in a lab baseline route, or no rows in strict default-deny mode.	Application-only authorization is fragile; default-deny is safer but not role-aware until configured.

After DDS	Own row plus direct reports; restricted columns based on manager grant.	Own row only.	Same prompt and same SQL become role-aware inside the database.
-----------	---	---------------	---

Before DDS:

Signed In User

username

indradarshni

sub

indradarshni

client_id

753fa24c1f864d9aab3135cae86e02ff

scope

all

group

["DB_Admns","DeepSec_Employee"]

grant type

azc

Ask The Agent

show me every employee salary and ssn

Run Query

Try Overpull

Planned SQL

attempt_overpull

```
SELECT employee_id,
first_name,
last_name,
job_code,
department_id,
ssn,
phone_number,
salary,
user_name,
manager_id
FROM hr.employees
ORDER BY employee_id
```

Returned Rows

employee_id	first_name	last_name	job_code	department_id	ssn	phone_number	salary	user_name	manager_id
1	Grace	Young	CEO		111-11-1111	555-100-0001	235000.0	grace	
2	Alex	Kovuru	SWE_MGR	1	222-22-2222	555-100-0002	175000.0	alexx	1
3	Indra	Darshni	SWE2	1	333-33-3333	555-100-0003	120000.0	indradarshni	2
4	Charlie	Davis	SWE1	1	444-44-4444	555-100-0004	95000.0	charlie	2
5	Dana	Lee	SWE3	1	555-55-5555	555-100-0005	130000.0	dana	2
6	Bob	Smith	SALES_REP	2	666-66-6666	555-100-0006	145000.0	bob	1
7	Fiona	Chen	HR_REP	3	777-77-7777	555-100-0007	92000.0	fiona	1

Signed In User

username

alexx

sub

alexx

client_id

753fa24c1f864d9aab3135cae86e02ff

scope

all

group

["DB_Users","DeepSec_Manager","DB_Admns","DeepSec_Employee","digiplatform_admins","ag-top-admins","Administrators","AG-ag-top-uprvk5g/rq_AG_Administrator"]

grant type

azc

Ask The Agent

show me every employee salary and ssn

Run Query

Try Overpull

Planned SQL

attempt_overpull

```
SELECT employee_id,
first_name,
last_name,
job_code,
department_id,
ssn,
phone_number,
salary,
user_name,
manager_id
FROM hr.employees
ORDER BY employee_id
```

Returned Rows

employee_id	first_name	last_name	job_code	department_id	ssn	phone_number	salary	user_name	manager_id
1	Grace	Young	CEO		111-11-1111	555-100-0001	235000.0	grace	
2	Alex	Kovuru	SWE_MGR	1	222-22-2222	555-100-0002	175000.0	alexx	1
3	Indra	Darshni	SWE2	1	333-33-3333	555-100-0003	120000.0	indradarshni	2
4	Charlie	Davis	SWE1	1	444-44-4444	555-100-0004	95000.0	charlie	2
5	Dana	Lee	SWE3	1	555-55-5555	555-100-0005	130000.0	dana	2
6	Bob	Smith	SALES_REP	2	666-66-6666	555-100-0006	145000.0	bob	1
7	Fiona	Chen	HR_REP	3	777-77-7777	555-100-0007	92000.0	fiona	1

After DDS:

Signed In User

username

indradarshni

sub

indradarshni

client_id

753fa24c1f864d9aab3135cae86e02ff

scope

all

group

["DB_Admns","DeepSec_Employee"]

grant type

azc

Ask The Agent

show me every employee salary and ssn

Run Query

Try Overpull

Planned SQL

attempt_overpull

```
SELECT employee_id,
first_name,
last_name,
job_code,
department_id,
ssn,
phone_number,
salary,
user_name,
manager_id
FROM hr.employees
ORDER BY employee_id
```

Returned Rows

employee_id	first_name	last_name	job_code	department_id	ssn	phone_number	salary	user_name	manager_id
3	Indra	Darshni	SWE2	1	333-33-3333	555-100-0003	120000.0	indradarshni	2

Signed In User

username

alexx

sub

alexx

client_id

753fa24c1f864d9aab3135cae86e02ff

scope

all

group

["DB_Users","DeepSec_Manager","DB_Admns","DeepSec_Employee","digiplatform_admins","ag-top-admins","Administrators","AG-ag-top-uprvk5g/rq_AG_Administrator"]

grant type

azc

Ask The Agent

show me every employee salary and ssn

Run Query

Try Overpull

Planned SQL

attempt_overpull

```
SELECT employee_id,
first_name,
last_name,
job_code,
department_id,
ssn,
phone_number,
salary,
user_name,
manager_id
FROM hr.employees
ORDER BY employee_id
```

Returned Rows

employee_id	first_name	last_name	job_code	department_id	ssn	phone_number	salary	user_name	manager_id
2	Alex	Kovuru	SWE_MGR	1	222-22-2222	555-100-0002	175000.0	alexx	1
3	Indra	Darshni	SWE2	1		555-100-0003	120000.0	indradarshni	2
4	Charlie	Davis	SWE1	1		555-100-0004	95000.0	charlie	2
5	Dana	Lee	SWE3	1		555-100-0005	130000.0	dana	2

20. Troubleshooting

Symptom	Likely cause	Fix
group is blank	Custom claim missing, named user_groups, old token, or user has no group	Create claim named group, use incognito, login again, verify OCI IAM group assignment.
ORA-00942 on HR.EMPLOYEES	No active data role or mandatory grants block service account	Confirm group claim, IAM mapped roles, data grants, and group names match exactly.
ORA-52601 token expired	OCI IAM access token timed out	Logout and login again.
ORA-52602 token parse/verify error	Wrong app_id or domain_url in DB config	Use DB Resource Application ID and include :443 in domain URL.
invalid_scope	Wrong OCI_DB_SCOPE	Use full DB resource scope, for example https://db26ai:2484/DB26AI_PDB1a II.
TCPS timeout	Network security rule or private IP path issue	Verify compute can reach DB private IP port 2484.
Shell says malformed URL	Smart quotes pasted into curl	Use plain ASCII quotes only.